

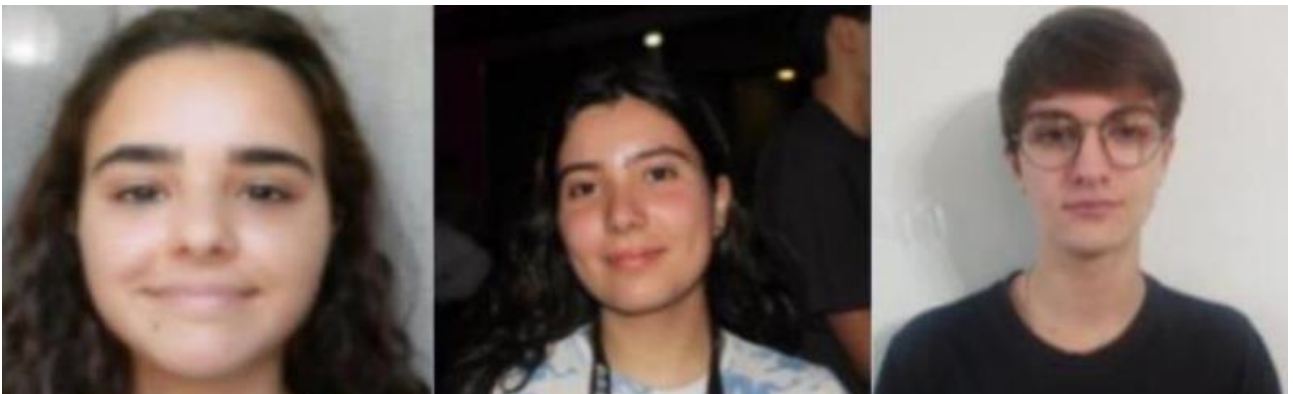
Universidade do Minho



Computação Gráfica

Grupo 16

Fase 4



Eduarda Vieira, A104098

Leonor Cunha, A103997

Pedro Rebelo, A104091

Índice

1. Introdução	3
2. Gerador - Normais e Coordenadas de Textura	3
2.1 Função auxiliar writeVertex.....	3
2.2 Plano	3
2.3 Caixa	3
2.4 Esfera	4
2.5 Cone	4
2.6 Superfícies de Bézier	4
3. Engine - Formato de Vértice Intercalado	5
3.1 Deduplicação de vértices	5
4. Engine - Iluminação	5
4.1 Modelo de iluminação Phong.....	5
4.2 Fontes de luz	6
4.3 Configurações globais de iluminação.....	6
5. Engine - Mapeamento de Texturas.....	7
5.1 Carregamento com DevIL.....	7
5.2 Parâmetros de textura	7
5.3 Aplicação por modelo.....	7
6. Parser XML - Extensões da Fase 4	7
6.1 Elemento model.....	7
6.2 Elemento color	8
6.3 Elemento lights.....	8
6.4 Chave de identificação de modelos	8
7. Cena de Demonstração - Sistema Solar.....	8
7.1 Estrutura hierárquica.....	8
7.2 Fonte de luz	8
7.3 Materiais por corpo celeste	8
7.4 Texturas utilizadas.....	9
8. Câmara Orbital em Coordenadas Esféricas	9
9. Compilação e Execução.....	10
Compile Generator	10
Compile Engine	10
Gerar primitivas	10
Executar a cena do sistema solar.....	10
10. Decisões de Implementação.....	10
11. Conclusão.....	11

1. Introdução

A Fase 4 conclui o motor 3D desenvolvido ao longo das fases anteriores, adicionando as duas funcionalidades que mais aproximam a cena de uma renderização realista: iluminação e mapeamento de texturas. Sobre a base da Fase 3, que já disponibilizava VBOs com índices, animações temporais e superfícies de Bézier, esta fase estende o gerador de primitivas para calcular normais e coordenadas de textura por vértice, e estende o engine para carregar texturas, configurar fontes de luz e aplicar o modelo de iluminação Phong via OpenGL fixo.

As três contribuições principais são:

1. extensão do gerador para emitir normais analíticas e coordenadas UV por primitiva;
2. suporte completo a iluminação e texturas no engine, usando DevIL para carregamento de imagens e um layout de vértice intercalado com 8 floats;
3. cena de demonstração do sistema solar animado com texturas reais e uma luz pontual no centro do Sol.

2. Gerador - Normais e Coordenadas de Textura

O gerador foi alargado para que cada vértice emitido no ficheiro .3d inclua, além da posição, a normal unitária e as coordenadas de textura. Esta informação é calculada analiticamente em tempo de geração, pelo que o engine não precisa de a recalcular em tempo de execução.

2.1 Função auxiliar writeVertex

Para garantir consistência de formato entre todas as primitivas, foi criada uma função auxiliar writeVertex que recebe os 8 atributos e escreve uma linha XML com o formato:

Qualquer primitiva que produza um triângulo chama esta função para cada um dos três vértices, eliminando duplicação de código e assegurando que o engine encontra sempre os mesmos atributos, independentemente da primitiva.

2.2 Plano

O plano é gerado no plano XZ, pelo que a normal de todos os vértices é constante e apontada para cima: (0, 1, 0). As coordenadas de textura são obtidas por interpolação linear da grelha: para um vértice na coluna j e linha i de uma grelha de $\text{divisions} \times \text{divisions}$, tem-se $s = j / \text{divisions}$ e $t = i / \text{divisions}$. Desta forma, a textura cobre uniformemente toda a superfície do plano.

2.3 Caixa

A caixa tem seis faces, cada uma com uma normal constante apontando para o exterior. As normais estão codificadas numa tabela fixa de seis vetores:

Face	Normal
Z+ (frente)	(0, 0, 1)
Z- (trás)	(0, 0, -1)
Y+ (topo)	(0, 1, 0)
Y- (base)	(0, -1, 0)

Face	Normal
X- (esquerda)	(-1, 0, 0)
X+ (direita)	(1, 0, 0)

As coordenadas de textura são calculadas localmente dentro de cada face, variando de (0, 0) no canto inferior esquerdo até (1, 1) no canto superior direito. Cada face recebe o mapeamento completo da textura, pelo que uma textura aplicada à caixa aparece repetida em todas as seis faces.

2.4 Esfera

A normal de um ponto na superfície de uma esfera centrada na origem é o vetor posição normalizado: $n = (x, y, z) / r$. Este resultado decorre diretamente da geometria esférica: o gradiente da equação da esfera $x^2 + y^2 + z^2 = r^2$ é $2(x, y, z)$, que após normalização dá exatamente o vetor radial. Assim, em vez de calcular o produto externo, basta dividir a posição pelo raio:

$$n_x = x / r \quad n_y = y / r \quad n_z = z / r$$

As coordenadas de textura usam o mapeamento esférico standard baseado nos ângulos paramétricos:

- $s = \phi / (2\pi)$, onde ϕ é o ângulo de longitude, que varia de 0 a 1 ao longo de um paralelo;
- $t = \theta / \pi$, onde θ é o ângulo polar, que varia de 0 no polo norte a 1 no polo sul.

Este mapeamento é o mais comum para planetas e globos porque distribui a textura de forma intuitiva: latitude e longitude da textura correspondem a latitude e longitude da superfície.

2.5 Cone

A normal lateral de um cone requer atenção especial porque a superfície não é plana nem esférica. Para um cone de raio r e altura h , a normal num ponto da lateral numa direção angular θ é proporcional a $(\cos\theta, r/h, \sin\theta)$. A componente Y é r/h porque corresponde ao seno do ângulo de abertura do cone: quanto mais aberto o cone, mais a normal aponta para cima. Após normalização:

$$\text{Vec3 } n = \text{normalize}(\{\cos(\theta), \text{slopeY}, \sin(\theta)\});$$

onde $\text{slopeY} = r/h$.

Para o ápice do cone, a normal é calculada como média das normais dos dois vértices da base do triângulo, garantindo uma transição suave sem descontinuidade. A base do cone usa a normal (0, -1, 0) para todos os vértices, com mapeamento polar centrado em (0.5, 0.5).

2.6 Superfícies de Bézier

Nas superfícies paramétricas de Bézier bicúbicas, a normal em cada ponto é calculada pelo produto externo das derivadas parciais $\partial P / \partial u \times \partial P / \partial v$. Cada derivada é calculada usando o vetor de derivadas $[3t^2, 2t, 1, 0]$ em vez do vetor de avaliação $[t^3, t^2, t, 1]$:

$$\text{Vec3 } dU = \text{bezierSurface_dU}(u, v, \text{patch})$$

$$\text{Vec3 } dV = \text{bezierSurface_dV}(u, v, \text{patch})$$

$$\text{Vec3 } n = \text{normalize}(\text{cross}(dU, dV))$$

As coordenadas de textura usam mapeamento direto dos parâmetros da superfície: $s = u$, $t = v$. Este mapeamento é o mais natural para superfícies paramétricas porque respeita a estrutura interna do patch.

3. Engine - Formato de Vértice Intercalado

O formato do vértice foi estendido da Fase 3, que usava apenas posição, para 8 floats que incluem também a normal e as coordenadas de textura:

Offset (floats)	Atributo	Componentes
0	Posição	x, y, z
3	Normal	nx, ny, nz
6	Textura	s, t

A decisão de usar um layout intercalado, em vez de arrays separados para cada atributo, deve-se à eficiência de cache: quando a GPU processa um vértice, todos os seus atributos estão em memória contígua, minimizando cache misses. O stride é de 8 floats, ou 32 bytes.

No momento do upload para a GPU, os dados são aplanados num array flat de floats e enviados num único `glBufferData`. Os três pointers são configurados com o mesmo stride e offsets distintos:

```
glVertexPointer(3, GL_FLOAT, 8 * sizeof(float), (void*)0)
```

```
glNormalPointer(GL_FLOAT, 8 * sizeof(float), (void*)(3 * sizeof(float)))
```

```
glTexCoordPointer(2, GL_FLOAT, 8 * sizeof(float), (void*)(6 * sizeof(float)))
```

3.1 Deduplicação de vértices

Na Fase 3, a chave de deduplicação era apenas a posição. Na Fase 4, a chave inclui todos os 8 atributos. Isto é necessário porque o mesmo ponto no espaço pode ter normais diferentes dependendo da face a que pertence. Por exemplo, numa aresta de uma caixa, o mesmo ponto geométrico surge em duas faces com normais perpendiculares. Se a chave fosse apenas a posição, um dos dois vértices sobreporia o outro, produzindo normais incorretas e iluminação errada.

4. Engine - Iluminação

4.1 Modelo de iluminação Phong

O modelo de iluminação de Phong decompõe a luz refletida por uma superfície em quatro componentes independentes, que são somadas para obter a cor final do fragmento:

Componente	Descrição	Parâmetro no XML
Difusa	Luz que depende do ângulo entre a normal e a direção da luz	diffuse R G B
Ambiente	Iluminação mínima, independente da luz	ambient R G B
Especular	Reflexo brilhante dependente do ângulo de visualização	specular R G B

Componente	Descrição	Parâmetro no XML
Emissiva	Cor própria do objeto, independente de qualquer luz	emissive R G B

Os valores RGB no XML estão em [0, 255]. O parser normaliza-os para [0, 1] antes de os passar ao OpenGL via `glMaterialfv`. O brilho especular é controlado pelo parâmetro `shininess`, entre 0 e 128: valores elevados concentram o brilho numa área pequena, enquanto valores baixos espalham o brilho por uma área maior.

4.2 Fontes de luz

O enunciado requer suporte a três tipos de fonte de luz, que se distinguem pela forma como o OpenGL interpreta o quarto componente, `W`, do vetor de posição:

Tipo	W	Característica
point	1.0	Luz pontual com posição; irradia em todas as direções
directional	0.0	Luz direcional, infinitamente distante; sem atenuação
spot	1.0	Cone de luz com posição, direção e ângulo de corte

As luzes são configuradas na função `applyLights`, chamada no início de cada frame após `gluLookAt`. Esta ordem é essencial: em OpenGL, a posição de uma luz é transformada pela matriz `modelview` no momento em que `glLightfv` é chamada. Ao posicioná-la após `gluLookAt` mas antes de qualquer transformação de objeto, a luz fica no espaço do mundo, que é o comportamento desejado.

O OpenGL suporta até 8 luzes simultâneas, de `GL_LIGHT0` a `GL_LIGHT7`. Todas são configuradas com cor branca para difusa e especular, e uma ambiente fraca de 30% para evitar sombras completamente negras. Para a spotlight, o expoente de concentração foi definido como 2.0, produzindo um feixe moderadamente concentrado.

4.3 Configurações globais de iluminação

Além das fontes de luz individuais, foram ativadas as seguintes configurações globais no engine:

- `GL_LIGHTING`: ativa o cálculo de iluminação global;
- `GL_COLOR_MATERIAL`: permite que `glColor3f` continue a funcionar com iluminação ativa;
- `GL_NORMALIZE`: normaliza automaticamente as normais após transformações geométricas, essencial quando `scale` não é uniforme;
- `glLightModelfv(GL_LIGHT_MODEL_AMBIENT)`: luz ambiente global fraca, com valor 0.1, como iluminação de fundo.

5. Engine - Mapeamento de Texturas

5.1 Carregamento com DevIL

O carregamento de imagens usa a biblioteca DevIL, conforme recomendado na unidade curricular. A opção pelo DevIL em detrimento de bibliotecas alternativas como stb_image justifica-se pela disponibilidade no ambiente de compilação e pela integração direta com o formato de dados esperado pelo OpenGL.

O processo de carregamento segue os seguintes passos:

1. inicialização do DevIL com `ilInit` e configuração da origem da textura para o canto inferior esquerdo;
2. carregamento da imagem para memória com `ilLoadImage`;
3. conversão para formato RGBA com `ilConvertImage`;
4. upload para a GPU via `glTexImage2D` e geração de mipmaps com `glGenerateMipmap`;
5. libertação da memória DevIL com `ilDeleteImages`.

Para evitar carregar a mesma imagem duas vezes, foi implementado um cache de texturas, com a chave sendo o caminho do ficheiro.

5.2 Parâmetros de textura

As texturas são configuradas com os seguintes parâmetros:

- `GL_TEXTURE_WRAP_S / GL_TEXTURE_WRAP_T = GL_REPEAT`: a textura repete quando as coordenadas são exteriores a `[0, 1]`;
- `GL_TEXTURE_MAG_FILTER = GL_LINEAR`: interpolação bilinear em close-up para evitar pixelização;
- `GL_TEXTURE_MIN_FILTER = GL_LINEAR`: interpolação para texturas afastadas.

O modo de combinação é `GL_MODULATE`, que multiplica a cor da textura pelos valores de iluminação calculados pelo Phong. Desta forma, uma região da textura em sombra fica mais escura, e uma região iluminada fica com a cor real da textura.

5.3 Aplicação por modelo

Cada modelo pode ter uma textura diferente, ou nenhuma. A lógica de desenho ativa e desativa a textura consoante o modelo.

Se um modelo não tem textura, apenas o material de cor é aplicado, o que permite misturar modelos texturizados e não texturizados na mesma cena.

6. Parser XML - Extensões da Fase 4

6.1 Elemento model

Cada elemento `model` pode conter um sub-elemento `texture` com um atributo `file`. O parser lê o caminho do ficheiro e armazena-o na estrutura `Model`. A resolução do caminho tenta várias

localizações alternativas, como o diretório do XML e o subdiretório textures, para facilitar a organização dos ficheiros.

6.2 Elemento color

O elemento color pode conter os sub-elementos diffuse, ambient, specular, emissive e shininess. Os valores RGB são lidos como inteiros em [0, 255] e normalizados para [0, 1] pelo parser. Se o elemento color não estiver presente, o material usa valores padrão: diffuse cinzento claro, ambient escuro, specular zero, emissive zero e shininess zero.

6.3 Elemento lights

O elemento lights é lido diretamente dentro de world. Suporta três tipos:

As luzes são armazenadas no rootGroup para acesso direto no engine. O valor padrão de cutoff é 180 graus, que faz a spotlight equivaler a uma luz pontual e garante retrocompatibilidade com cenas sem cutoff explícito.

6.4 Chave de identificação de modelos

Na Fase 3, os modelos eram identificados apenas pelo caminho do ficheiro .3d. Na Fase 4, a chave é modelFile + '|' + texFile. Esta alteração permite que o mesmo ficheiro geométrico seja usado com texturas diferentes, sem que uma sobrescreva a outra no mapa de dados.

7. Cena de Demonstração - Sistema Solar

7.1 Estrutura hierárquica

A cena usa a mesma hierarquia do sistema solar dinâmico da Fase 3, estendida com texturas reais e materiais diferenciados por planeta. O nó raiz é o Sol, que funciona como âncora de todos os planetas. Cada planeta está num grupo filho do Sol, com a transformação rotate time seguida de translate e scale. As luas estão em grupos filhos dos respetivos planetas, herdando o movimento orbital do planeta.

7.2 Fonte de luz

A cena tem uma única fonte de luz pontual posicionada na origem, que coincide com o centro do Sol. Esta escolha simula fisicamente o comportamento do sistema solar real: a luz emana do Sol e diminui com a distância. A luz pontual permite que os planetas mais próximos apareçam mais iluminados e os mais distantes mais escuros.

7.3 Materiais por corpo celeste

Cada corpo celeste foi configurado com um material que reflete as suas propriedades físicas reais:

Corpo	Textura	Decisão de material
Sol	2k_sun.jpg	emissive alto, para emitir luz própria
Mercúrio	2k_mercury.jpg	cinzento, specular mínimo
Vénus	2k_venus_surface.jpg	amarelado, specular moderado

Corpo	Textura	Decisão de material
Terra	2k_earth_daymap.jpg	azul-esverdeado, specular moderado
Marte	2k_mars.jpg	avermelhado, specular muito baixo
Júpiter	2k_jupiter.jpg	alaranjado, diffuse intenso
Saturno	2k_saturn.jpg	amarelado pálido, anel sem textura de cor própria
Urano	2k_uranus.jpg	azul-esverdeado claro
Netuno	2k_neptune.jpg	azul intenso, specular mínimo

O Sol merece destaque especial: como emite luz própria, a sua aparência não deve depender da posição da única fonte de luz da cena, que é ele próprio. Por isso, o emissive foi definido como uma cor quente e brilhante, e o ambient foi igualado ao diffuse para que o Sol apareça sempre completamente iluminado, independentemente do ângulo da câmara.

7.4 Texturas utilizadas

As texturas dos planetas foram obtidas do Solar System Scope, conforme sugerido no enunciado. Os ficheiros estão em formato JPEG a 2K de resolução, e os anéis de Saturno usam uma textura PNG com canal alfa para a transparência.

8. Câmara Orbital em Coordenadas Esféricas

A câmara foi implementada como câmara orbital, usando coordenadas esféricas α , β , r , onde α é o azimuth horizontal, β é a elevação e r é a distância ao centro da cena. Esta representação é ideal para orbitar em torno de um objeto porque a rotação é feita por simples adição de ângulos, sem necessidade de multiplicar matrizes ou resolver sistemas de equações.

A conversão de coordenadas esféricas para cartesianas é feita com:

$$\text{posX} = \text{lookAtX} + r * \sin(\alpha) * \cos(\beta)$$

$$\text{posY} = \text{lookAtY} + r * \sin(\beta)$$

$$\text{posZ} = \text{lookAtZ} + r * \cos(\alpha) * \cos(\beta)$$

O ângulo β é limitado ao intervalo $[-\pi/2 + \epsilon, \pi/2 - \epsilon]$ para evitar o gimbal lock que ocorreria quando a câmara passasse pelos polos. Os controlos de teclado são:

Tecla	Ação
↑ / ↓	Rotação vertical (β)
← / →	Rotação horizontal (α / azimuth)
W / S	Zoom in / out (raio r)
A	Toggle eixos 3D
L	Toggle wireframe
ESC	Terminar

9. Compilação e Execução

As instruções de compilação e execução assumem Linux com as bibliotecas freeglut, DevIL e OpenGL instaladas.

Compilar Generator

```
g++ -std=c++17 generator.cpp -o generator
```

Compilar Engine

```
g++ engine.cpp camera.cpp parser.cpp tinyxml2.cpp -o engine -lglut -lGL -lGLU -lIL
```

Gerar primitivas

```
./generator sphere 1 30 30 sphere.3d
```

```
./generator plane 2 10 plane.3d
```

```
./generator box 1 3 box.3d
```

```
./generator cone 1 2 8 4 cone.3d
```

Executar a cena do sistema solar

```
./engine xmlFiles/solar_system.xml
```

10. Decisões de Implementação

Esta secção reúne as decisões de implementação mais relevantes, explicando o raciocínio por trás de cada escolha.

DevIL vs stb_image: o carregamento de imagens usa DevIL em vez de stb_image porque o ambiente de desenvolvimento da cadeira já tem DevIL disponível. Além disso, DevIL suporta um vasto leque de formatos sem configuração adicional e a sua API é consistente com o pipeline OpenGL fixo usado no projeto.

GL_NORMALIZE vs normalização manual: ativar GL_NORMALIZE garante que as normais permanecem unitárias mesmo quando o objeto está dentro de um grupo com escala não unitária. A alternativa seria usar GL_RESCALE_NORMAL, que é mais eficiente mas só correto com escala uniforme, ou normalizar manualmente no gerador para a escala final. Dado que a cena usa escalas diversas por planeta, GL_NORMALIZE é a opção mais robusta.

Origem da textura IL_ORIGIN_LOWER_LEFT: o OpenGL assume que a coordenada $t = 0$ corresponde ao canto inferior da textura, mas muitos formatos de imagem guardam os píxeis do topo para baixo. Ao configurar IL_ORIGIN_LOWER_LEFT, o DevIL inverte automaticamente a imagem, evitando que as texturas apareçam de cabeça para baixo.

Chave modelo + textura no mapa: a mudança de chave de modelFile para modelFile + '|' + texFile foi motivada pela necessidade de reutilizar a mesma geometria com diferentes texturas para planetas diferentes. Sem esta mudança, o carregamento do segundo planeta com a mesma esfera sobreporia os dados do primeiro.

Luz após gluLookAt: as luzes são configuradas no início de cada frame, após gluLookAt e antes de

qualquer transformação de objeto. Em OpenGL, a posição da luz é transformada pelo estado da matriz modelview no momento de `glLightfv`. Se a luz fosse configurada antes de `gluLookAt`, ficaria no espaço da câmara e acompanharia os movimentos da câmara, um comportamento incorreto para o sistema solar.

11. Conclusão

A Fase 4 completou o ciclo de desenvolvimento do motor 3D, adicionando as duas funcionalidades que mais contribuem para o realismo visual: iluminação e mapeamento de texturas. O resultado é uma cena interativa do sistema solar onde cada planeta é reconhecível pela sua textura e o modelo de iluminação Phong dá profundidade e relevo a toda a geometria.

Do ponto de vista técnico, as principais contribuições desta fase são: a extensão do formato de vértice para um layout intercalado de 8 floats, que unifica geometria, normais e coordenadas UV num único VBO; o cálculo analítico de normais para cada primitiva, que garante iluminação correta sem recorrer a aproximações; e a integração do DevIL para carregamento de texturas, com cache para evitar duplicação de uploads.

A cena de demonstração valida todas as funcionalidades em conjunto: a luz pontual no centro do Sol ilumina diferentemente cada planeta consoante a sua distância e ângulo; as texturas reais tornam cada planeta imediatamente identificável; e os materiais diferenciados capturam características físicas como a emissão do Sol, o brilho especular dos oceanos da Terra e a superfície mate de Marte. A hierarquia de grupos da Fase 2, as animações da Fase 3 e a renderização da Fase 4 funcionam em conjunto de forma coerente, concretizando o objetivo global do trabalho: um mini motor 3D baseado em scene graph com capacidades de renderização realista.